VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



CO3010 - Microprocessors-Microcontrollers

LAB REPORT

Lab 3 - Buttons/Switches

Advisor(s): Phan Văn Sỹ
Student(s): Nguyễn Đức Phi Long ID 2452689
Class ID: CC04

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Github **4**

2 Problems Encountered **4**

3 Exercise and Report **4**

3.1 Specifications 4
3.2 Exercise 1: Sketch an FSM 5
3.3 Exercise 2: Proteus Schematic 7
3.4 Exercise 3: Create STM32 Project 7
3.5 Exercise 4: Modify Timer Parameters 8
3.6 Exercise 5: Adding code for button debouncing 12
3.7 Exercise 6: Adding code for displaying modes 19
3.8 Exercise 7: Adding code for increasing time duration value for the red LEDs 23
3.9 Exercise 8: Adding code for increasing time duration value for the amber LEDs 27
3.10 Exercise 9: Adding code for increasing time duration value for the green LEDs 28
3.11 Exercise 10: To finish the project 28

1 Github

- Link for every exercises in Lab 3: Link
- Link for the testing video: Link

2 Problems Encountered

- The PDF file didn't mention `HAL_TIM_Base_Start_IT(&htim2);` in `main.c`. Remember to initialize it.

- In the lab session, we encounter lag when simulating on Proteus8. If you see the same problem, we recommend to change inside tab `System -> Set Animation Options`:

- Frames per Second: 20 (by default) -> 10
- Timestep per Frame: 50m (default) -> 100m

3 Exercise and Report

3.1 Specifications

You are required to build an application of a traffic light in a cross road which includes some features as described below:

- The application has 12 LEDs including 4 red LEDs, 4 amber LEDs, 4 green LEDs.
- The application has 4 seven segment LEDs to display time with 2 for each road. The 2 seven segment LEDs will show time for each color LED corresponding to each road.
- The application has three buttons which are used
  - to select modes,
  - to modify the time for each color led on the fly, and
  - to set the chosen value.
- The application has at least 4 modes which is controlled by the first button. Mode 1 is a normal mode, while modes 2 3 4 are modification modes. You can press the first button to change the mode. Modes will change from 1 to 4 and back to 1 again.



Mode 1 - Normal mode:

- The traffic light application is running normally.

Mode 2 - Modify time duration for the red LEDs: This mode allows you to change the time duration of the red LED in the main road. The expected behaviours of this mode include:

- All single red LEDs are blinking in 2 Hz.

- Use two seven-segment LEDs to display the value.

- Use the other two seven-segment LEDs to display the mode.

- The second button is used to increase the time duration value for the red LEDs.

- The value of time duration is in a range of 1 - 99.

- The third button is used to set the value.

Mode 3 - Modify time duration for the amber LEDs: Similar for the red LEDs described above with the amber LEDs.

Mode 4 - Modify time duration for the green LEDs: Similar for the red LEDs described above with the green LEDs.

3.2 Exercise 1: Sketch an FSM

Your task in this exercise is to sketch an FSM that describes your idea of how to solve the problem.

Please add your report here.

- In this exercise, we will notate:

- BTN1: button 1 (same for the two others)
- R1/R2: Road 1/Road 2

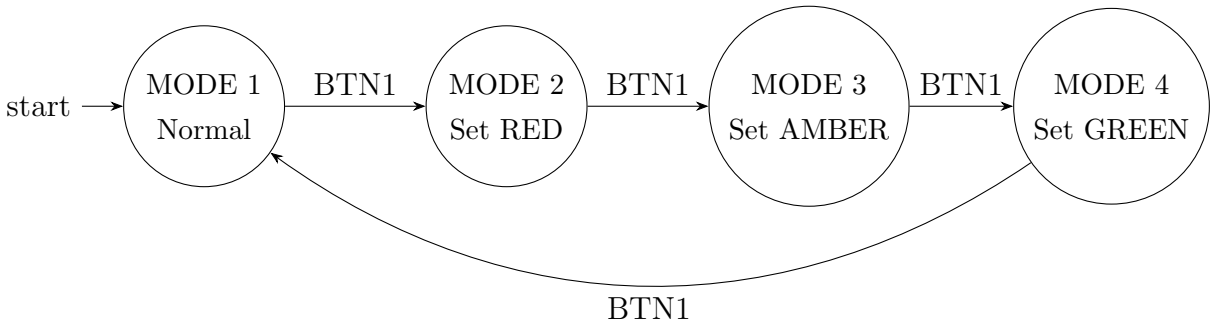


Figure 3.1: Mode FSM for Button 1

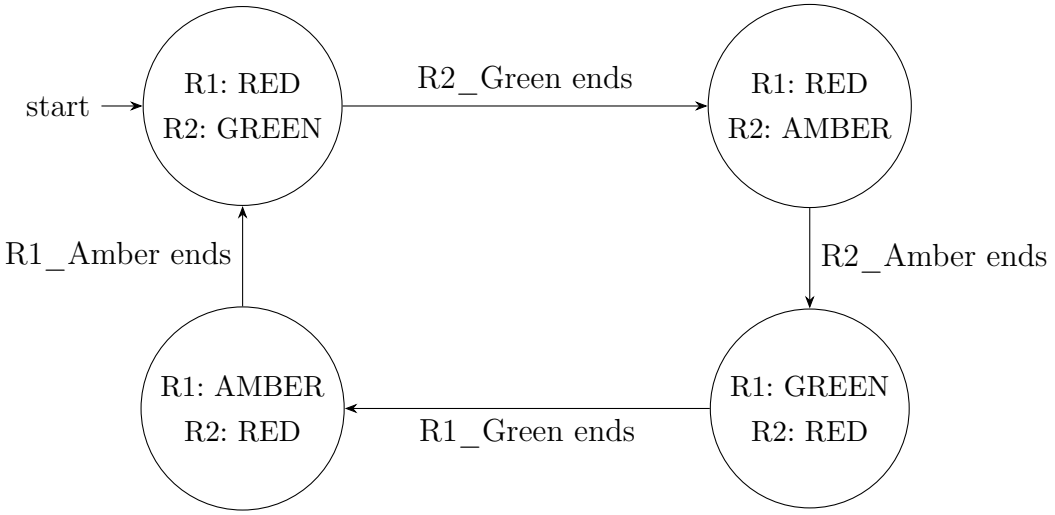


Figure 3.2: Traffic light FSM in Mode 1

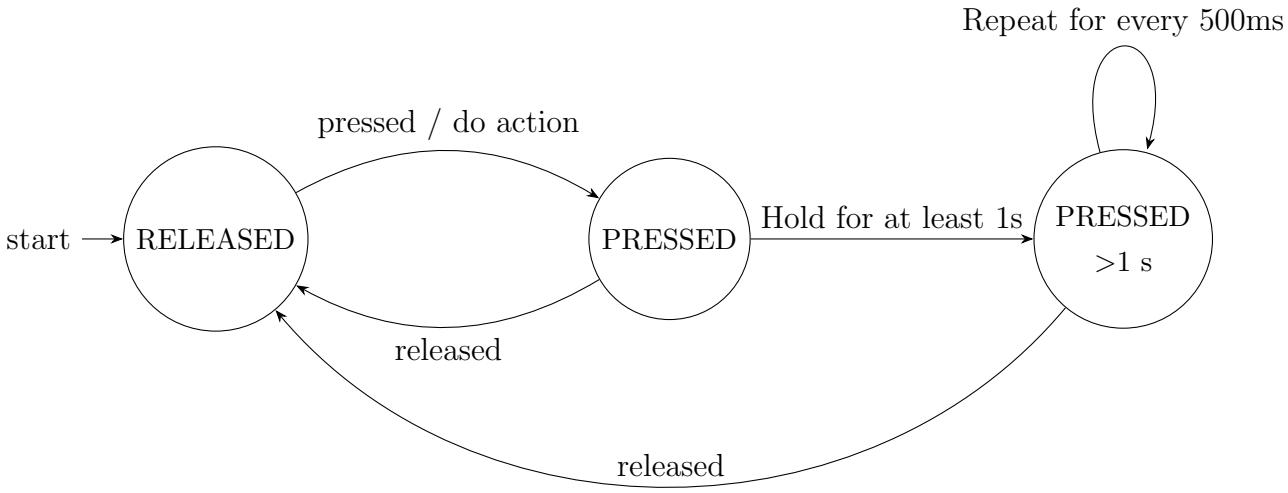


Figure 3.3: Button FSM for mechanic in general

3.3 Exercise 2: Proteus Schematic

Your task in this exercise is to draw a Proteus schematic for the problem above. Please add your report here.

Note: we will use this schematic for this whole lab session.

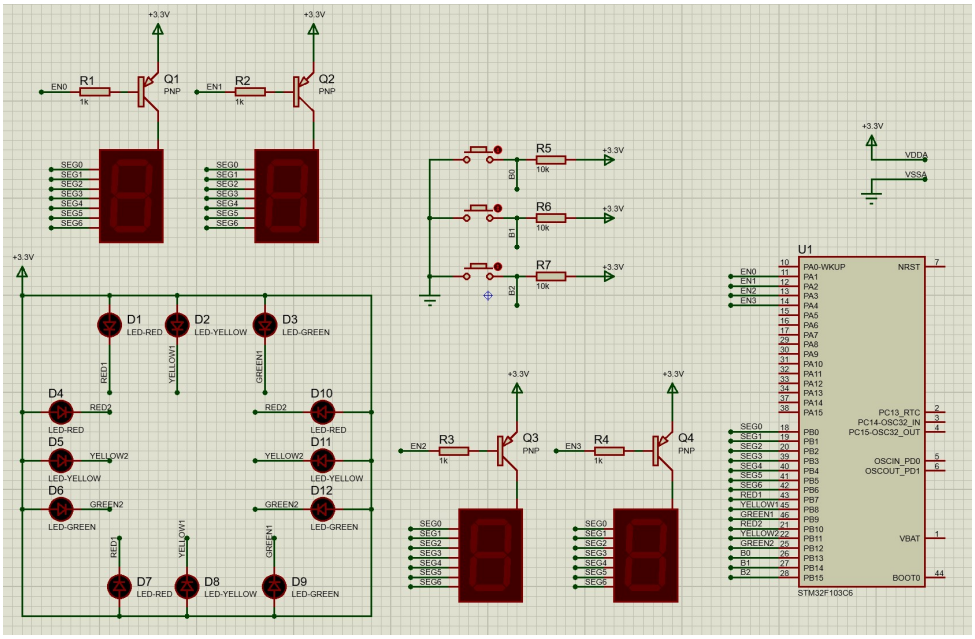


Figure 3.4: *Proteus schematic for a formal 4-way traffic*

3.4 Exercise 3: Create STM32 Project

Your task in this exercise is to create a project that has pin corresponding to the Proteus schematic that you draw in previous section. You need to set up your timer interrupt is about 10ms.

Please add your report here.

- We will initialize the necessay codes in the Exercise 4, other settings for the 10ms timer interrupt stay the same as the 2nd Lab session:

- HCLK = 8 MHz
- Prescaler = 7999
- Counter Period = 9

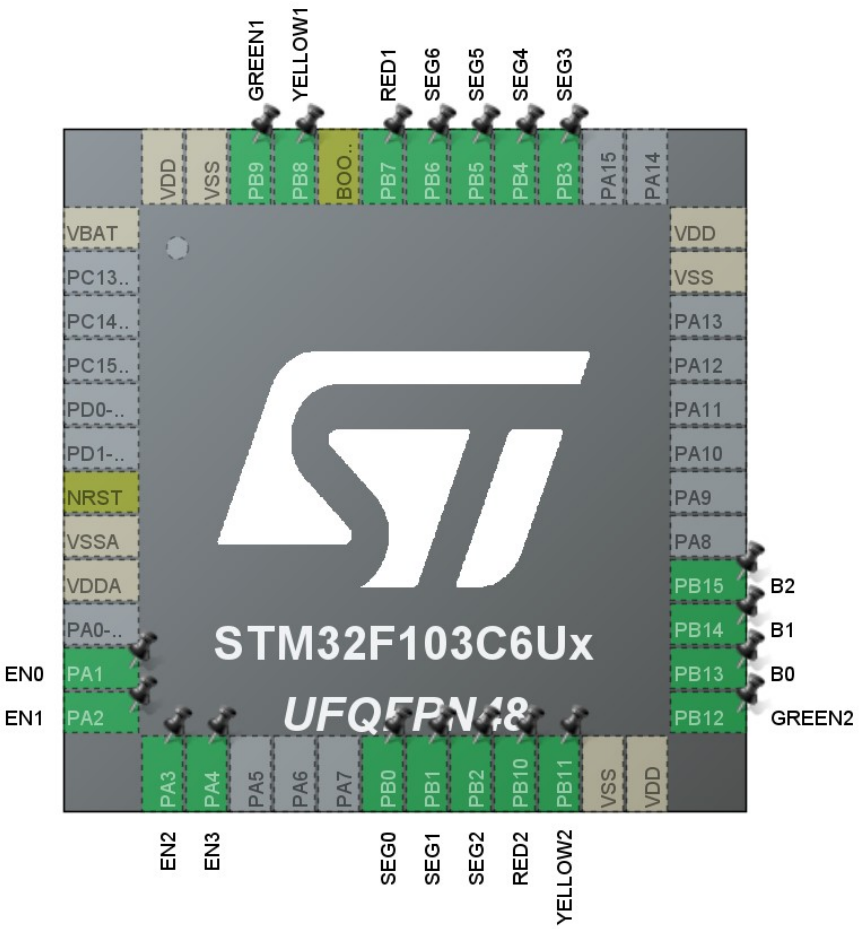


Figure 3.5: *STM32CubeIDE schematic for a formal 4-way traffic*

3.5 Exercise 4: Modify Timer Parameters

Your task in this exercise is to modify the timer settings so that when we want to change the time duration of the timer interrupt, we change it the least and it will not affect the overall system. For example, the current system we have implemented is that it can blink an LED in 2 Hz, with the timer interrupt duration is 10ms. However, when we want to change the timer interrupt duration to 1ms or 100ms, it will not affect the 2Hz blinking LED.

Please add your report here.

- To do the exercise, we will implement the two files `timer.h` and `timer.c`.

Listing 3.1: Initial implementation for timer.h

```
#ifndef INC_TIMER_H_
#define INC_TIMER_H_

#include "main.h"

// Match the TIM2 interrupt period (in ms)
#define TIMER_CYCLE   1

// Timer indexes
#define TIMER_TRAFFIC 0 // 1 s tick for countdown
#define TIMER_BLINK   1 // 250 ms toggle -> 2 Hz blink
#define TIMER_SCAN    2 // 7-seg digit scanning
#define TIMER_AUTOINC 3 // BTN2 auto-repeat (500 ms)
#define MAX_TIMERS    4

void setTimer(int index, int duration_ms);
int  isTimerExpired(int index);
void timerRun(void);
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim);

#endif /* INC_TIMER_H_ */
```

Listing 3.2: Add-on for timer.c

```
#include "timer.h"
#include "input_reading.h"

static int timerCounter[MAX_TIMERS];
static int timerFlag[MAX_TIMERS];

void setTimer(int index, int duration_ms)
{
    if (index < 0 || index >= MAX_TIMERS) // Safety check
      return;
    timerCounter[index] = duration_ms / TIMER_CYCLE;
```

```
12
13
14      if (timerCounter[index] < 1)       // Safety check
15        timerCounter[index] = 1;       // in case of [
              duration_ms] too low, e.g. 5/10 = 0, we set a
              minimum = 1
16      timerFlag[index] = 0;
17  }
18
19
20  int isTimerExpired(int index)
21  {
22      if (index < 0 || index >= MAX_TIMERS) return 0;
23      if (timerFlag[index])
24      {
25          timerFlag[index] = 0;        // clear on read: one
                event per expiry
26          return 1;
27      }
28      return 0;
29  }
30
31
32  void timerRun(void)
33  {
34      for (int i = 0; i < MAX_TIMERS; i++)
35      {
36          if (timerCounter[i] > 0)
37          {
38              timerCounter[i]--;
39              if (timerCounter[i] == 0) timerFlag[i] = 1;
40          }
41      }
42  }
43
```

```
44
45 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
46 {
47     if (htim->Instance == TIM2)
48     {
49         button_reading();
50         timerRun();
51     }
52 }
```

- For confirmation when modifying Timer, we can use this example code below and change the relevant parameters (remember to update the #include in `main.c` as it resets whenever you make changes in file `.ioc`).

- The code can be used for both cases.

Table 3.1: Configuration for different Timer Interrupt

Timer Interrupt	Prescaler	Counter Period	TIMER_CYCLE
1 ms	799	9	1
10 ms	7999	9	10

Listing 3.3: Temporary code for Exercise 4 test

```
1 /* Includes
    ------------------------------------------------------------------
    */
2 #include "main.h"
3 #include "timer.h"
4
5
6 int main(void)
7 {
8     MX_TIM2_Init();
9     MX_GPIO_Init();
```

```
10
11      /* USER CODE BEGIN 2 */
12      HAL_TIM_Base_Start_IT(&htim2);
13      setTimer(TIMER_BLINK, 250);
14      /* USER CODE END 2 */
15
16      while (1)
17      {
18          if (isTimerExpired(TIMER_BLINK))    // temporary test
19          {
20              setTimer(TIMER_BLINK, 250);
21              HAL_GPIO_TogglePin(RED1_GPIO_Port, RED1_Pin);
22          }
23      }
24  }
```

3.6 Exercise 5: Adding code for button debouncing

Following the example of button reading and debouncing in the previous section, your tasks in this exercise are:

- To add new files for input reading and output display,
- To add code for button debouncing,
- To add code for increasing mode when the first button is pressed.

Please add your report here.

- For Exercise 5, we will adjust these files:

- `input_processing.h`
- `input_reading.c`: add buttons ports and pins to read
- `input_processing.c`: process the input button and return mode
- `led_display.h`
- `led_display.c`: display the 7-segment LED base on the mode



- `main.c`: change the testing codes to recent exercise

- We also optimize some codes from the initial initialization but not changing its algorithm. Thus, we will present only what are added to the files.
- The test is expected to change modes by pressing button B0 and display the mode on the 7-segment LED EN0.

Listing 3.4: input_reading.c

```
1 //we aim to work with more than one buttons
2 #define NO_OF_BUTTONS     3
3
4 //timer interrupt duration is 10ms, so to pass 1 second,
5 //we need to jump to the interrupt service routine 100 time
6 #define DURATION_FOR_AUTO_INCREASING  (1000 / TIMER_CYCLE)
7 #define BUTTON_IS_PRESSED       GPIO_PIN_RESET
8 #define BUTTON_IS_RELEASED        GPIO_PIN_SET
9
10 //the buffer that the final result is stored after debouncing
11 static GPIO_PinState buttonBuffer[NO_OF_BUTTONS]       = {
   GPIO_PIN_SET, GPIO_PIN_SET, GPIO_PIN_SET };;
12
13 //we define two buffers for debouncing
14 static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS] = {
    GPIO_PIN_SET, GPIO_PIN_SET, GPIO_PIN_SET };;
15 static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS] = {
    GPIO_PIN_SET, GPIO_PIN_SET, GPIO_PIN_SET };;
16
17 //we define a flag for a button pressed more than 1 second.
18 static uint8_t flagForButtonPress1s[NO_OF_BUTTONS];
19
20 //we define a counter for automatically increasing the value
21 //after the button is pressed more than 1 second.
22 static uint16_t counterForButtonPress1s[NO_OF_BUTTONS];
23
24 // From ex05
25 static GPIO_TypeDef *const  btnPort[NO_OF_BUTTONS]  = {
```

```
    B0_GPIO_Port, B1_GPIO_Port, B2_GPIO_Port };
26 static const uint16_t    btnPin[NO_OF_BUTTONS] = { B0_Pin,
    B1_Pin, B2_Pin };
27
28
29 void button_reading(void)
30 {
31     for(char i = 0; i < NO_OF_BUTTONS; i++)
32     {
33         debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
34         debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(btnPort[i
            ], btnPin[i]);
35
36         if(debounceButtonBuffer1[i] == debounceButtonBuffer2[
            i])
37         {
38             buttonBuffer[i] = debounceButtonBuffer1[i];
39
40             if(buttonBuffer[i] == BUTTON_IS_PRESSED)
41             {
42                 //if a button is pressed, we start counting
43                 if(counterForButtonPress1s[i] <
                    DURATION_FOR_AUTO_INCREASING)
44                 {
45                     counterForButtonPress1s[i]++;
46                 }
47                 else
48                 {
49                     //the flag is turned on when 1 second has
                        passed
50                     //since the button is pressed.
51                     flagForButtonPress1s[i] = 1;
52                 }
53             }
54             else
```



```
55              {
56                  counterForButtonPress1s[i] = 0;
57                  flagForButtonPress1s[i] = 0;
58              }
59          }
60      }
61  }
```

Listing 3.5: input_processing.h

```
1  #ifndef INC_INPUT_PROCESSING_H_
2  #define INC_INPUT_PROCESSING_H_
3
4  #define MODE_NORMAL   1
5  #define MODE_RED    2
6  #define MODE_AMBER    3
7  #define MODE_GREEN    4
8  #define MODE_MAX    4
9
10 void fsm_for_input_processing(void);
11 int  get_mode(void);
12
13 #endif /* INC_INPUT_PROCESSING_H_ */
```

Listing 3.6: input_processing.c

```
1  // From ex05
2  static int mode = MODE_NORMAL;
3
4  int get_mode(void)
5  {
6      return mode;
7  }
8
9  void fsm_for_input_processing(void)
10 {
```

```
    switch(buttonState)
    {
    case BUTTON_RELEASED:
      if(is_button_pressed(0))
      {
        buttonState = BUTTON_PRESSED;
        mode++;              // INCREASE VALUE OF PORT A BY
           ONE UNIT

        if (mode > MODE_MAX)
          mode = MODE_NORMAL;
      }
      break;


    case BUTTON_PRESSED:
      if(!is_button_pressed(0))
      {
        buttonState = BUTTON_RELEASED;
      }
      else if(is_button_pressed_1s(0))
      {
        buttonState = BUTTON_PRESSED_MORE_THAN_1_SECOND;
      }
      break;


    case BUTTON_PRESSED_MORE_THAN_1_SECOND:
      if(!is_button_pressed(0))
      {
        buttonState = BUTTON_RELEASED;
      }
      break;
    }
}
```

Listing 3.7: led_display.h

```h
#ifndef INC_LED_DISPLAY_H_
#define INC_LED_DISPLAY_H_

void display7SEG(int num);

#endif /* INC_LED_DISPLAY_H_ */
```

Listing 3.8: led_display.c

```c
#include "main.h"
#include "led_display.h"

typedef struct
{
    GPIO_TypeDef *port;
    uint16_t pin;
} SegPin;

SegPin segments[] =
{
    { SEG0_GPIO_Port, SEG0_Pin },
    { SEG1_GPIO_Port, SEG1_Pin },
    { SEG2_GPIO_Port, SEG2_Pin },
    { SEG3_GPIO_Port, SEG3_Pin },
    { SEG4_GPIO_Port, SEG4_Pin },
    { SEG5_GPIO_Port, SEG5_Pin },
    { SEG6_GPIO_Port, SEG6_Pin },
};

GPIO_PinState digitPatterns[10][7] =
{
    { RESET, RESET, RESET, RESET, RESET, RESET, SET   }, // 0
    { SET,   RESET, RESET, SET,   SET,   SET,   SET   }, // 1
```

```
25      { RESET, RESET, SET,   RESET, RESET, SET,   RESET }, //
            2
26      { RESET, RESET, RESET, RESET, SET,   SET,   RESET }, //
            3
27      { SET,   RESET, RESET, SET,   SET,   RESET, RESET }, //
            4
28      { RESET, SET,   RESET, RESET, SET,   RESET, RESET }, //
            5
29      { RESET, SET,   RESET, RESET, RESET, RESET, RESET }, //
            6
30      { RESET, RESET, RESET, SET,   SET,   SET,   SET   }, //
            7
31      { RESET, RESET, RESET, RESET, RESET, RESET, RESET }, //
            8
32      { RESET, RESET, RESET, RESET, SET,   RESET, RESET }, //
            9
33  };
34
35  void display7SEG(int num)
36  {
37      if (num < 0 || num > 9) return;
38
39    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_SET);
40    HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_SET);
41    HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_SET);
42    HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_SET);
43
44      for (int i = 0; i < 7; i++)
45          HAL_GPIO_WritePin(segments[i].port, segments[i].pin,
              digitPatterns[num][i]);
46  }
```

Listing 3.9: main.c

```
1 #include "main.h"
2 #include "timer.h"
```

```
#include "input_processing.h"
#include "led_display.h"

MX_TIM2_Init();
MX_GPIO_Init();

/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);
static int lastMode = 0;
/* USER CODE END 2 */

while (1)
{
    //you only need to add the fsm function here
    fsm_for_input_processing();

    if (get_mode() != lastMode)
    {
        lastMode = get_mode();
        display7SEG(lastMode);
        HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin,
           GPIO_PIN_RESET);
    }
}
```

3.7 Exercise 6: Adding code for displaying modes

Your tasks in this exercise are:

- To add code for display mode on seven-segment LEDs, and
- To add code for blinking LEDs depending on the mode that is selected.

Please add your report here.

- For Exercise 6, we will adjust these files:

- `input_processing.h`

- `input_processing.c`: add duration for the traffic lights and 7-segment LEDs
- `led_display.h`
- `led_display.c`: display the modes on EN0/EN1 and the duration of the conrresponding traffic lights on EN2/EN3, all through out the function `void fsm_for_output_processing(void)`
- `main.c`: change the testing codes to recent exercise

Listing 3.10: input_processing.h

```
int get_duration(int index);              // 0 = red, 1 = amber
    , 2 = green
```

Listing 3.11: input_processing.c

```
int get_duration(int index)
{
    if (index < 0 || index > 2) return 0;
    return duration[index];
}
```

Listing 3.12: led_display.h

```
#ifndef INC_LED_DISPLAY_H_
#define INC_LED_DISPLAY_H_

void display7SEG(int num);

#endif /* INC_LED_DISPLAY_H_ */
```

Listing 3.13: led_display.h

```
void update_buffer(int mode, int value);  // fills the 4-
    digit buffer
void scan_leds(void);             // show the next digit
void leds_all_off(void);
```

```
4 void leds_toggle_color(int color);       // 0 = red, 1 = amber
    , 2 = green (both roads)
5
6 void output_init(void);
7 void fsm_for_output_processing(void);
```

Listing 3.14: led_display.c

```
1 void update_buffer(int mode, int value)
2 {
3     led_buffer[0] = mode / 10;
4     led_buffer[1] = mode % 10;
5     led_buffer[2] = value / 10;
6     led_buffer[3] = value % 10;
7 }
8
9 void scan_leds(void)
10 {
11     display7SEG(led_buffer[index_led]);                    //
          turns all EN off, writes the segments
12     HAL_GPIO_WritePin(enPort[index_led], enPin[index_led],
          GPIO_PIN_RESET); // enable one digit
13     index_led = (index_led + 1) % 4;
14 }
15
16 void leds_all_off(void)                                  // active-
    low: SET = off
17 {
18     for (int c = 0; c < 3; c++)
19         for (int r = 0; r < 2; r++)
20             HAL_GPIO_WritePin(ledPort[c][r], ledPin[c][r],
                  GPIO_PIN_SET);
21 }
22
23 void leds_toggle_color(int color)
24 {
```

```
    if (color < 0 || color > 2) return;
    for (int r = 0; r < 2; r++)
        HAL_GPIO_TogglePin(ledPort[color][r], ledPin[color][r
            ]);
}

void output_init(void)
{
    leds_all_off();
    setTimer(TIMER_SCAN, SCAN_PERIOD_MS);
}

void fsm_for_output_processing(void)
{
    static int lastMode = 0;
    int m = get_mode();

    if (m != lastMode)
    {
        lastMode = m;
        leds_all_off();
        setTimer(TIMER_BLINK, BLINK_PERIOD_MS);
    }


    if (m == MODE_NORMAL)
      update_buffer(m, 0);
    else
      update_buffer(m, get_duration(m - MODE_RED));


    if (isTimerExpired(TIMER_SCAN))
    {
        setTimer(TIMER_SCAN, SCAN_PERIOD_MS);
        scan_leds();
```

```
59      }
60
61
62      if (m != MODE_NORMAL && isTimerExpired(TIMER_BLINK))
63      {
64          setTimer(TIMER_BLINK, BLINK_PERIOD_MS);
65          leds_toggle_color(m - MODE_RED);
66      }
67  }
```

Listing 3.15: main.c

```
1 HAL_TIM_Base_Start_IT(&htim2);
2 output_init();
3
4 while (1)
5 {
6     //you only need to add the fsm function here
7     fsm_for_input_processing();
8     fsm_for_output_processing();
9 }
```

3.8 Exercise 7: Adding code for increasing time duration value for the red LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the red LEDs
- to use the third button to set the value for the red LEDs.

Please add your report here.

- For Exercise 7, we will adjust these files:

- `input_processing.h`

- `input_processing.c`: add `static void button_action(int index)` and update `fsm_for_input_processing(void)`
- `led_display.c`: slightly additional change

Listing 3.16: input_processing.h

```
int get_temp(void);        // value currently being edited
```

Listing 3.17: input_processing.c

```
int get_temp(void) { return tempValue; }

// What each button does when it is pressed (or auto-repeated)
static void button_action(int index)
{
    switch (index)
    {
    case 0:      // B0: next mode
      mode++;
      if (mode > MODE_MAX)
        mode = MODE_NORMAL;


      if (mode == MODE_NORMAL)
          tempValue = 0;
      else
          tempValue = duration[mode - MODE_RED];
      break;


    case 1:      // B1: increase temp value
      if (mode != MODE_NORMAL)
      {
        tempValue++;
        if (tempValue > 99)
```



```
26            tempValue = 1;
27        }
28        break;
29
30
31      case 2:      // B2: set (save) the value
32        if (mode != MODE_NORMAL)
33          duration[mode - MODE_RED] = tempValue;
34        break;
35      }
36  }
37
38  void fsm_for_input_processing(void)
39  {
40      for (int i = 0; i < NUM_BUTTONS; i++)
41      {
42          switch (buttonState[i])
43          {
44        case BUTTON_RELEASED:
45          if (is_button_pressed(i))
46          {
47            buttonState[i] = BUTTON_PRESSED;
48            button_action(i);
49          }
50          break;
51
52
53        case BUTTON_PRESSED:
54          if (!is_button_pressed(i))
55          {
56            buttonState[i] = BUTTON_RELEASED;
57          }
58          else if (is_button_pressed_1s(i))
59          {
60            buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
```

```
61
62            if (i == 1)
63              setTimer(TIMER_AUTOINC, AUTO_REPEAT_MS);
64          }
65          break;
66
67
68        case BUTTON_PRESSED_MORE_THAN_1_SECOND:
69          if (!is_button_pressed(i))
70          {
71            buttonState[i] = BUTTON_RELEASED;
72          }
73          else if (i == 1 && isTimerExpired(TIMER_AUTOINC))
              // auto-repeat only for B1
74          {
75            setTimer(TIMER_AUTOINC, AUTO_REPEAT_MS);
76            button_action(1);
77          }
78          break;
79          }
80      }
81  }
```

Listing 3.18: led_display.c

```
1  void fsm_for_output_processing(void)
2  {
3      /*
4      ...
5      */
6
7      if (m == MODE_NORMAL)
8        update_buffer(m, 0);
9      else
10       update_buffer(m, get_temp());
11
```

```
12
13      if (isTimerExpired(TIMER_SCAN))
14      /*
15      ...
16      */
17  }
```

3.9 Exercise 8: Adding code for increasing time duration value for the amber LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the amber LEDs
- to use the third button to set the value for the amber LEDs.

Please add your report here.

- Modifying the amber duration reuses the mechanism built in Exercise 7, so no new code is needed. The color being edited is not hard-coded: it is derived from the current mode as `mode - MODE_RED`, which gives index 0 for red (Mode 2), 1 for amber (Mode 3) and 2 for green (Mode 4). The three saved durations are stored in a single array `duration[3]`, initialized to 5, 2 and 3 seconds for red, amber and green respectively.

- When the first button (B0) moves the system into Mode 3, a temporary value `tempValue` is loaded from the saved amber duration (`duration[1]`). The second button (B1) increases `tempValue` by one on each press and wraps from 99 back to 1, so the value always stays in the range 1–99. If B1 is held for more than one second, the button FSM enters the `BUTTON_PRESSED_MORE_THAN_1_SECOND` state and the value increases automatically every 500 ms. The third button (B2) copies `tempValue` into `duration[1]`, which saves the new amber duration. If the user leaves the mode without pressing B2, the edit is discarded, because the next entry into the mode reloads `tempValue` from the saved value.

- For the display, the two seven-segment LEDs on EN0 and EN1 show the mode number (`03`) and the two LEDs on EN2 and EN3 show `tempValue`. The four digits are multiplexed by scanning one digit every 10 ms. While in this mode, only the amber LEDs of both roads blink at 2 Hz, using a software timer that toggles them every 250 ms; the other colors stay off.

3.10 Exercise 9: Adding code for increasing time duration value for the green LEDs

Your tasks in this exercise are:

- to use the second button to increase the time duration value of the green LEDs
- to use the third button to set the value for the green LEDs.

Please add your report here.

- Modifying the green duration works exactly like the amber and red cases, since all three share the same code. In Mode 4 the index `mode - MODE_RED` evaluates to 2, so the button actions operate on `duration[2]`, whose default value is 3 seconds.

3.11 Exercise 10: To finish the project

Your tasks in this exercise are:

- To integrate all the previous tasks to one final project
- To create a video to show all features in the specification
- To add a report to describe your solution for each exercise.
- To submit your report and code on the BKeL

- For the final and complete project, we will adjust only one file `led_display.c` in order to display the real traffic cycle. Moreover, we have to handle the wrong button inputs (e.g. set duration for RED - AMBER - GREEN to 7/7/7 seconds respectively).

- By initializing hard constraint:

```
int gap = R - G - A;       // how much longer the saved red is
    than green + amber
if (gap < 0) gap = 0;      // red can't be shorter than that,
   so no negative gap
int redTime = G + A + gap;
```

we can let the traffic lights run smoothly despite the wrong inputs.

- For the detail codes, please check over `STM32CubeIDE/lab_03/Core/Src/led_display.c`.